

Dynamic Programming

Introduction

- Dynamic programming (Dynamic Planning) คือเทคนิคที่ใช้แก้ปัญหที่ประกอบด้วยปัญหาย่อยที่มีการซ้อนทับกัน
- โดยปกติ ปัญหาย่อยเกิดจาก recurrence ที่เชื่อมสัมพันธ์คำตอบของปัญหาหนึ่งกับคำตอบต่างๆของปัญหาย่อยๆ ประเภทเดียวกัน
- แทนการแก้ปัญหาย่อยๆ ซ้ำแล้วซ้ำเล่า dynamic programming ใช้วิธีแก้ปัญหาย่อยแต่ละปัญหาเพียงครั้งเดียวแล้วบันทึกคำตอบลงในตารางที่เราสามารถนำมาใช้หาคำตอบของปัญหาตั้งต้นได้

ตัวอย่าง Fibonacci

- ชุดตัวเลข Fibonacci 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, . . . , สามารถนิยามได้ด้วยสมการ recurrence

$$F(n) = F(n - 1) + F(n - 2) \quad \text{for } n \geq 2$$

- โดยมีสมการตั้งต้น

$$F(0) = 0, \quad F(1) = 1.$$

- สังเกตได้ว่า ปัญหา $F(n)$ แบ่งออกเป็น 2 ปัญหาย่อยที่ซ้อนทับกันคือ $F(n-1)$ และ $F(n-2)$
- ถ้าเราพยายามแก้ใช้นิยาม recurrence เพื่อหาค่า $F(n)$ จะมีการคำนวณซ้ำๆ เกิดขึ้นมากมาย

Algorithm Fibonacci

ตัวอย่าง

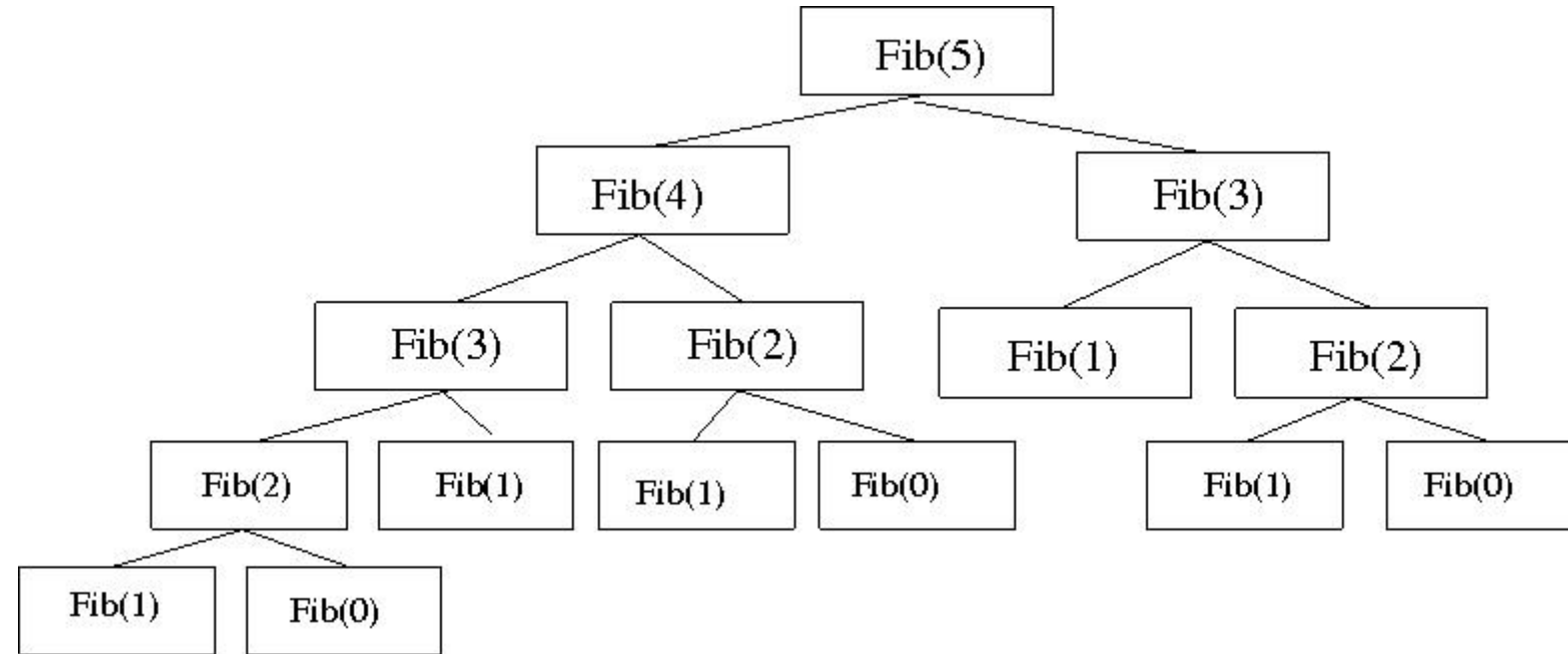
Fib (n)

if (n <= 1)

return n

return Fib(n-1) + Fib(n-2)

Calling Tree Fib(5)



Solution 1 : Top Down

- ซึ่งสามารถใช้อาร์เรย์เข้ามาช่วยได้ โดยเก็บค่าต่าง ๆ ของ $F(n)$ ในแต่ละช่องของอาร์เรย์

Fib (n)

if (n <= 1)

return M[n]

if M[n-2] is not calculated

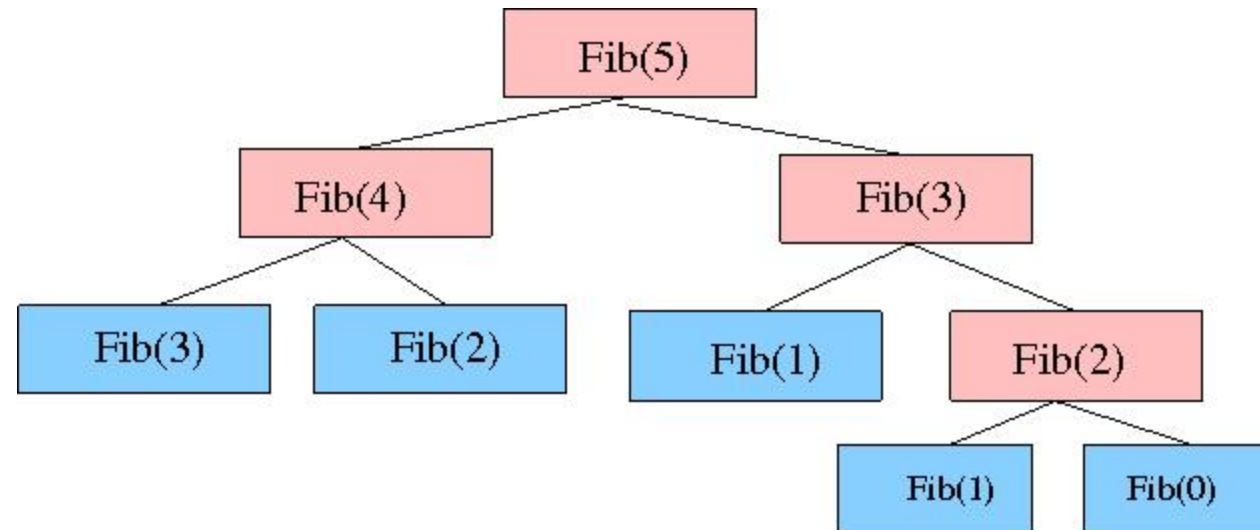
M[n-2] = Fib(n-2)

if M[n-1] is not calculated

M[n-1] = Fib(n-1)

return M[n-1] + M[n-2]

Calling Tree of Top Down DP Fib(n)



Solution 2 : Bottom Up

Fib (n)

$A[0] = 0; A[1] = 1;$

for $i=1$ to n do

$A[i] = A[i-1] + A[i-2]$

return $A[n]$

Solution 2 Improvement

- นอกจากนี้ เราสามารถหลีกเลี่ยงการใช้อาร์เรย์ได้ด้วยการใช้ตัวแปรเพียงสองตัวเพื่อเก็บข้อมูลล่าสุด 2 ตัวที่ได้จากการคำนวณ

Fib (n)

```
a = 0; b = 1;
```

```
for i=1 to n do
```

```
    c = a + b;
```

```
    a = b;
```

```
    b = c;
```

```
return c;
```

สรุป

- เมื่อพบปัญหาที่มีการคำนวณซ้ำซ้อน เทคนิคแบบ Dynamic programming สามารถช่วยลดเวลาลงไปได้ แต่จะเกิดการสิ้นเปลือง memory มากขึ้น
- เทคนิค dynamic programming สามารถแบ่งออกได้เป็น 2 รูปแบบ คือ bottom-up และ top-down
- แต่สิ่งที่สำคัญที่สุดคือ การหาสมการ recurrence ที่เชื่อมสัมพันธ์คำตอบของปัญหากับ คำตอบของปัญหาย่อยที่ซ้อนทับกัน

Content

1. Binomial Coefficient
2. Optimal Binary Search Tree
3. Knapsack Problem
4. Longest Common Subsequence

1. Computing a Binomial Coefficient

- **binomial coefficient** $C(n, k)$ or $\binom{n}{k}$ คือจำนวนของ subsets ของข้อมูล k ชุด จากเซตที่มีข้อมูลทั้งหมด n ชุด ($0 \leq k \leq n$).

- ชื่อ binomial coefficients มาจากสมการ binomial

$$(a + b)^n = C(n, 0)a^n + C(n, 1)a^{n-1}b + C(n, 2)a^{n-2}b^2 + \dots + C(n, k)a^{n-k}b^k + \dots + C(n, n)b^n$$

- คุณสมบัติสองข้อของ binomial coefficients ที่เราสนใจคือ

$$C(n, k) = C(n - 1, k - 1) + C(n - 1, k) \text{ for } n > k > 0$$

และ

$$C(n, 0) = C(n, n) = 1$$

การคำนวณค่า Binomial Coefficient

	0	1	2	...	k-1	k
0	1					
1	1	1				
2	1	2	1			
⋮						
k	1					1
⋮						
n-1	1				$C(n-1, k-1)$	$C(n-1, k)$
n	1					$C(n, k)$

- ใช้ตารางขนาด $n + 1$ แถว $k + 1$ คอลัมน์
- ในการคำนวณ ให้เติมตารางทีละแถว เริ่มจากแถว 0 จนถึงแถว n ในแต่ละแถวให้เติมจากซ้ายไปขวา เริ่มจาก 1 เพราะ $C(n, 0) = 1$

Computing a Binomial Coefficient

Algorithm Binomial(n, k)

// Input : A pair of nonnegative integers $n \geq k \geq 0$

// Output : The value of $C(n, k)$

for $i=0$ to n do

 for $j=0$ to $\text{Min}(i,k)$ do

 if ($j=0$ or $j=i$) then

$C[i,j] \leftarrow 1$

 else $C[i,j] \leftarrow C[i-1, j-1] + C[i-1, j]$

return $C[n,k]$

Computing a Binomial Coefficient

- basic operation คือ การบวก ดังนั้นให้ $A(n, k)$ คือ จำนวนรวมของการบวกที่เกิดขึ้นในการคำนวณ $C(n, k)$.
- ข้อมูลใน $k + 1$ แถวแรกของตารางมีลักษณะเป็น 3 เหลี่ยม ในขณะที่ แถวที่เหลืออีก $n-k$ แถวมีลักษณะเป็น 4 เหลี่ยม
- เราสามารถแบ่งผลรวม $A(n, k)$ ออกเป็นสองส่วน

$$\begin{aligned} A(n, k) &= \sum_{i=1}^k \sum_{j=1}^{i-1} 1 + \sum_{i=k+1}^n \sum_{j=1}^k 1 = \sum_{i=1}^k (i-1) + \sum_{i=k+1}^n k \\ &= \frac{(k-1)k}{2} + k(n-k) \in \Theta(nk). \end{aligned}$$

Content

1. Binomial Coefficient

2. Optimal Binary Search Tree

3. Knapsack Problem

4. Longest Common Subsequence

2. Optimal Binary Search Trees

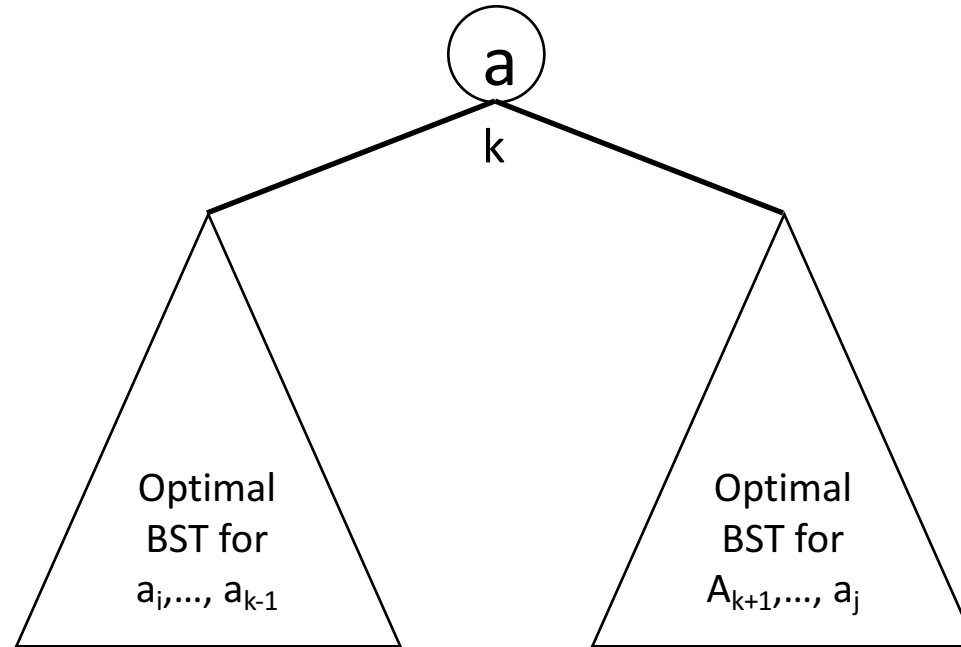
- OBST คือ BST ที่มีรูปแบบที่ส่งผลให้จำนวนครั้งของ basic operation น้อยที่สุด
 - เช่น การเปรียบเทียบ ในการ search
 - อาศัยข้อมูลในอดีต ทำให้สามารถทราบความน่าจะเป็นของการทำงานในแต่ละโหนด
- ตัวอย่าง พิจารณาคีย์ 4 ตัว คือ A, B, C และ D ให้ความน่าจะเป็นในการค้นหาคีย์ทั้ง 4 ตัวเป็นดังนี้ 0.1, 0.2, 0.4, 0.3 ตามลำดับ BST แบบใดให้การทำงานดีที่สุด
- เราสามารถหา optimal tree ได้โดยการสร้าง binary search trees ทั้งหมดของคีย์ 4 ตัวนี้ (14 ต้น) มาจาก Catalan number ลำดับที่ n

$$c(n) = \frac{2n}{n+1} \frac{1}{n} \quad \text{for } n \geq 0, c(0) = 1$$

Optimal Binary Search Trees

- กำหนดให้ $a_1, \dots, a_n$ คือ คีย์ที่แตกต่างกันและเรียงลำดับจากน้อยไปมาก
- กำหนดให้ $p_1, \dots, p_n$ คือ ความน่าจะเป็นของการค้นหาคีย์เหล่านี้
- กำหนดให้ $C[i, j]$ คือ average number of comparisons ที่น้อยที่สุดที่ใช้ในการค้นหาคีย์เจอใน binary tree T_i^j ที่ประกอบด้วยคีย์ $a_i, \dots, a_j$, โดย i, j คือ integer indices, $1 \leq i \leq j \leq n$.

Optimal Binary Search Trees



- BST ที่มี root a_k และมี optimal binary search subtrees T_i^{k-1} และ T_{k+1}^j เป็นลูก

Optimal Binary Search Trees

$$C[i, j] = \min_{i \leq k \leq j} \left\{ p_k \cdot 1 + \sum_{s=i}^{k-1} p_s \cdot (\text{level of } a_s \text{ in } T_i^{k-1} + 1) + \sum_{s=k+1}^j p_s \cdot (\text{level of } a_s \text{ in } T_{k+1}^j + 1) \right\}$$

$$C[i, j] = \min_{i \leq k \leq j} \left\{ p_k + \sum_{s=i}^{k-1} p_s \cdot (\text{level of } a_s \text{ in } T_i^{k-1}) + \sum_{s=i}^{k-1} p_s + \sum_{s=k+1}^j p_s \cdot (\text{level of } a_s \text{ in } T_{k+1}^j) + \sum_{s=k+1}^j p_s \right\}$$

$$C[i, j] = \min_{i \leq k \leq j} \left\{ \sum_{s=i}^{k-1} p_s \cdot (\text{level of } a_s \text{ in } T_i^{k-1}) + \sum_{s=k+1}^j p_s \cdot (\text{level of } a_s \text{ in } T_{k+1}^j) + \sum_{s=i}^j p_s \right\}$$

$$C[i, j] = \min_{i \leq k \leq j} \{C[i, k-1] + C[k+1, j]\} + \sum_{s=i}^j p_s$$

Optimal Binary Search Trees

- Thus we have the recurrence

$$C[i, j] = \min_{i \leq k \leq j} \{C[i, k-1] + C[k+1, j]\} + \sum_{s=i}^j p_s \quad \text{for } 1 \leq i \leq j \leq n$$

- Note that this formula implies that

$$C[i, i] = p_i \text{ for } 1 \leq i \leq n,$$

$$C[2, 3] = \min_{k=2} C[2+1] + C[3, 3] + (p_2 + p_3) \\ k=3, C[2+2] + C[4, 3] + (p_2 + p_3)$$

Optimal Binary Search Trees

Table of the dynamic programming algorithm for constructing an optimal binary search tree

	0	1					j	n
1	0	p_1						goal
		0	p_2					
i							$C[i,j]$	
								p_n
$n+1$								0

Example 1

key	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
probability	0.1	0.2	0.4	0.3

- The initial tables look like this:

	main table				
	0	1	2	3	4
1	0	0.1			
2		0	0.2		
3			0	0.4	
4				0	0.3
5					0

	root table				
	0	1	2	3	4
1		1			
2			2		
3				3	
4					4
5					

Example 1

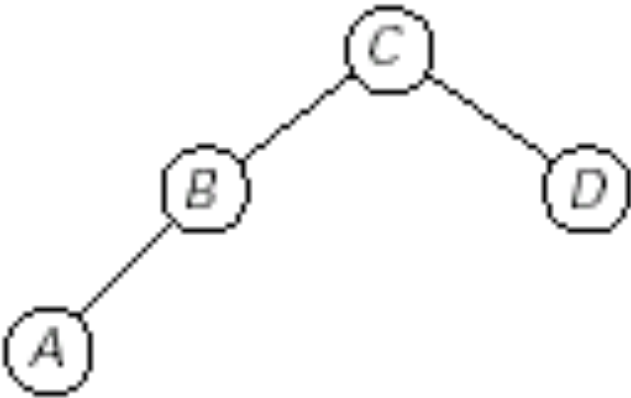
Let us compute $C[1, 2]$:

$$C[1, 2] = \min \begin{array}{l} k=1: C[1, 0] + C[2, 2] + \sum_{s=1}^2 p_s = 0 + 0.2 + 0.3 = 0.5 \\ k=2: C[1, 1] + C[3, 2] + \sum_{s=1}^2 p_s = 0.1 + 0 + 0.3 = 0.4 \end{array} = 0.4.$$

- จาก possible binary trees 2 ต้น root ของ optimal tree ควรจะมี index 2 (ก็คือ B) average number of comparisons ใน tree นี้เท่ากับ 0.4.

Example 1

main table						root table					
	0	1	2	3	4		0	1	2	3	4
1	0	0.1	0.4	1.1	1.7	1		1	2	3	3
2		0	0.2	0.8	1.4	2			2	3	3
3			0	0.4	1.0	3				3	3
4				0	0.3	4					4
5					0	5					



ALGORITHM *OptimalBST*($P[1..n]$)

//Finds an optimal binary search tree by dynamic programming

//Input: An array $P[1..n]$ of search probabilities for a sorted list of n keys

//Output: Average number of comparisons in successful searches in the

// optimal BST and table R of subtrees' roots in the optimal BST

for $i \leftarrow 1$ **to** n **do**

$C[i, i - 1] \leftarrow 0$

$C[i, i] \leftarrow P[i]$

$R[i, i] \leftarrow i$

$C[n + 1, n] \leftarrow 0$

for $d \leftarrow 1$ **to** $n - 1$ **do** //diagonal count

for $i \leftarrow 1$ **to** $n - d$ **do**

$j \leftarrow i + d$

$minval \leftarrow \infty$

for $k \leftarrow i$ **to** j **do**

if $C[i, k - 1] + C[k + 1, j] < minval$

$minval \leftarrow C[i, k - 1] + C[k + 1, j]; kmin \leftarrow k$

$R[i, j] \leftarrow k$

$sum \leftarrow P[i];$ **for** $s \leftarrow i + 1$ **to** j **do** $sum \leftarrow sum + P[s]$

$C[i, j] \leftarrow minval + sum$

return $C[1, n], R$

Analysis

- The algorithm's space efficiency is clearly quadratic
- The time efficiency of this version of the algorithm is cubic

Content

1. Binomial Coefficient
2. Optimal Binary Search Tree

3. Knapsack Problem

4. Longest Common Subsequence

3. Knapsack Problem

- Given n items:
 - weights: $w_1, w_2, \dots, w_n$
 - values: $v_1, v_2, \dots, v_n$
 - a knapsack of capacity W
- Find the most valuable subset of the items that fit into the knapsack

Example:

a knapsack of capacity 16

item	weight	value
1	2	\$20
2	5	\$30
3	10	\$50
4	5	\$10

Knapsack Problem

<i>Subset</i>	<i>Total weight</i>	<i>Total value</i>
{1}	2	\$20
{2}	5	\$30
{3}	10	\$50
{4}	5	\$10
{1,2}	7	\$50
{1,3}	12	\$70
{1,4}	7	\$30
{2,3}	15	\$80
{2,4}	10	\$40
{3,4}	15	\$60
{1,2,3}	17	not feasible
{1,2,4}	12	\$60
{1,3,4}	17	not feasible
{2,3,4}	20	not feasible
{1,2,3,4}	22	not feasible

Knapsack Problem

- To design a dynamic programming algorithm, we need to derive a recurrence relation that expresses a solution to an instance of the knapsack problem in terms of solutions to its smaller subinstances.
- ให้พิจารณา กรณีของ first i items, $1 \leq i \leq n$, with weights $w_1, \dots, w_i$, values $v_1, \dots, v_i$, and knapsack capacity j , $1 \leq j \leq W$.
- ให้ $V[i, j]$ คือค่าของ optimal solution ของกรณีนี้ นั่นคือค่าของ subset ที่มีค่ามากที่สุดของ first i items ที่สามารถใส่ลงในกระเป๋าที่มีความจุ j ได้
- เราสามารถแบ่ง subsets ทั้งหมดของ first i items ที่บรรจุลงในกระเป๋าได้เป็น 2 ประเภท คือ ประเภทที่ไม่มี i^{th} item และประเภทที่มี

จ. ๕ ๖ ๗

Knapsack Problem

- สำหรับ subset ที่ไม่มี i^{th} item ค่าของ optimal subset คือ $V[i - 1, j]$ ตามนิยาม
 - ในขณะที่ subset ที่มี i^{th} item (จะได้ว่า $j - w_i \geq 0$) optimal subset ประกอบด้วย
 - i^{th} item
 - optimal subset ของ first $i - 1$ items ที่สามารถบรรจุลงในกระเป๋าที่มีความจุ $j - w_i$.
 - $v_i + V[i - 1, j - w_i]$.
 - ดังนั้นค่าของ optimal solution ใน all feasible subsets ของ first i items คือ maximum ของทั้งสองค่าข้างบน
 - ถ้า i^{th} item ไม่สามารถบรรจุลงในกระเป๋าได้ ค่าของ optimal subset ของ first i items มีค่าเท่ากับค่าของ optimal subset ของ first $i - 1$ items.
- recurrence

Knapsack Problem

$$V[i, j] = \begin{cases} \max\{V[i-1, j], v_i + V[i-1, j-w_i]\} & \text{if } j-w_i \geq 0 \\ V[i-1, j] & \text{if } j-w_i < 0 \end{cases}$$

- initial conditions:

$$V[0, j] = 0 \text{ for } j \geq 0 \text{ and } V[i, 0] = 0 \text{ for } i \geq 0.$$

- เป้าหมายคือ การค้นหา $V[n, W]$, ค่ามากที่สุดของ subset ของ n items ที่สามารถบรรจุลงในกระเป๋าความจุ W

Table for Solving Knapsack Problem by Dynamic Programming

w_j, v_j

		0	$j-w_j$	j	w
0	0	0	0	0	0
i-1	0	$V[i-1, j-w_j]$	$V[i-1, j]$		
i	0		$V[i, j]$		
n	0				

i

Example

item	weight	value
1	2	\$12
2	1	\$10
3	3	\$20
4	2	\$15

capacity $W = 5$

capacity j

	i	0	1	2	3	4	5
$w_1 = 2, v_1 = 12$ $w_2 = 1, v_2 = 10$ $w_3 = 3, v_3 = 20$ $w_4 = 2, v_4 = 15$	0	0	0	0	0	0	0
	1	0	0	12	12	12	12
	2	0	10	12	22	22	22
	3	0	10	12	22	30	32
	4	0	10	15	25	30	37

Handwritten notes: Blue arrows and curly braces indicate the calculation of values in the table. A blue arrow points from the value 22 in row 3, column 3 to the value 25 in row 4, column 3. Another blue arrow points from the value 25 in row 4, column 3 to the value 30 in row 4, column 4. A blue curly brace groups the first three rows (i=1, 2, 3) and is labeled with the handwritten text "0 1 2 3".

Example

- maximal value คือ $V[4, 5] = \$37$
- เราสามารถหาว่าใน optimal subset ประกอบด้วย item ใดบ้าง โดยการ trace back การคำนวณในตาราง
 - เนื่องจาก $V[4, 5]$ ไม่เท่ากับ $V[3, 5]$ ดังนั้น item 4 ต้องอยู่ใน optimal subset $V[4, 5]$ แสดงว่า optimal subset ก่อนหน้านี้นี้ ที่มีความจุเหลือ $5 - 2 = 3$ คือ $V[3, 3]$
 - จาก $V[3, 3] = V[2, 3]$ แสดงว่า item 3 ไม่อยู่ใน optimal subset
 - เป็นต้น
- The time efficiency and space efficiency of this algorithm are both in $\Theta(nW)$.
- The time needed to find the composition of an optimal solution is in $O(n + W)$.

Memory Function

- The classic dynamic programming approach, fills a table with solutions to *all* smaller subproblems but each of them is solved only once.
- An unsatisfying aspect of this approach is that solutions to some of these smaller subproblems are often not necessary for getting a solution to the problem given.
- Since this drawback is not present in the top-down approach, it is natural to try to combine the strengths of the top-down and bottom-up approaches.
- The goal is to get a method that solves only subproblems that are necessary and does it only once. Such a method exists; it is based on using *memory functions*

Memory Function

- กำหนดค่าเริ่มต้นในตารางด้วย a special “null” symbol เพื่อบอกให้รู้ว่ายังไม่ได้มีการคำนวณในช่องนี้
- หลังจากนั้นเมื่อต้องการค่าใด ก็ตรวจสอบจากตารางว่า ได้มีการคำนวณมาแล้วหรือไม่
 - ถ้าคำนวณมาแล้ว ก็เอามาใช้ได้เลย
 - แต่ถ้ายังไม่ได้คำนวณก็คำนวณแล้วบันทึกค่าลงในตารางด้วย

Memory Function for solving Knapsack Problem

ALGORITHM *MFKnapsack*(i, j)

```
//Implements the memory function method for the knapsack problem
//Input: A nonnegative integer  $i$  indicating the number of the first
//       items being considered and a nonnegative integer  $j$  indicating
//       the knapsack's capacity
//Output: The value of an optimal feasible subset of the first  $i$  items
//Note: Uses as global variables input arrays Weights[1.. $n$ ], Values[1.. $n$ ]
//and table  $V[0..n, 0..W]$  whose entries are initialized with  $-1$ 's except for
//row 0 and column 0 initialized with 0's
if  $V[i, j] < 0$ 
    if  $j < \text{Weights}[i]$ 
         $value \leftarrow \text{MFKnapsack}(i - 1, j)$ 
    else
         $value \leftarrow \max(\text{MFKnapsack}(i - 1, j),$ 
                            $\text{Values}[i] + \text{MFKnapsack}(i - 1, j - \text{Weights}[i]))$ 
     $V[i, j] \leftarrow value$ 
return  $V[i, j]$ 
```


Memory Function for solving Knapsack Problem

		capacity j						
		i	0	1	2	3	4	5
		0	0	0	0	0	0	0
$w_1 = 2, v_1 = 12$	1	0	0	12	–	12	12	
$w_2 = 1, v_2 = 10$	2	0	–	12	22	–	22	
$w_3 = 3, v_3 = 20$	3	0	–	–	22	–	32	
$w_4 = 2, v_4 = 15$	4	0	–	–	–	–	37	

Memory Function

- In general, we cannot expect more than a constant-factor gain in using the memory function method for the knapsack problem because its time efficiency class is the same as that of the bottom-up algorithm
- A memory function method may be less space-efficient than a space efficient version of a bottom-up algorithm.

Content

1. Binomial Coefficient
2. Optimal Binary Search Tree
3. Knapsack Problem

4. Longest Common Subsequence

4. Longest Common Subsequence (LCS)

- Problem: Given sequences $x[1..m]$ and $y[1..n]$, find a **longest common subsequence** of both.
- กำหนดให้มีลำดับ 2 ชุดคือ $x[1..m]$ และ $y[1..n]$, จงหา ลำดับย่อยร่วมที่ยาวที่สุดระหว่าง x และ y
- Example: $x=ABC$ **BDAB** and $y=$ **BD****CABA**,
 - BCA is a common subsequence and
 - BCBA and **BDAB** are two LCSs

LCS Solution

- Brute force solution
- Writing a recurrence equation
- The dynamic programming solution

Brute force solution

- Solution: For every subsequence of x , check if it is a subsequence of y .
- สำหรับทุก ๆ ลำดับย่อยของ $x[1..m]$ ให้ตรวจสอบดูว่ามีปรากฏใน y หรือไม่
- บทวิเคราะห์:
 - จำนวนลำดับย่อยทั้งหมดของ x คือ 2^m
 - ในการตรวจสอบกับ y จะใช้เวลา $O(n)$ เนื่องจากเราต้อง scan y ทีละตัว เริ่มจากตัวที่ 1 ตามด้วย 2 ไปเรื่อย ๆ
 - ด้วยเหตุนี้ worst case running time จะอยู่ที่ $O(n2^m)$.

Writing the recurrence equation

- Let X_i denote the *ith prefix* $x[1..i]$ of $x[1..m]$, and
- X_0 denotes an empty prefix
- We will first compute the *length of an LCS of X_m and Y_n* , $LenLCS(m, n)$, and then use information saved during the computation for finding the actual subsequence
- We need a recursive formula for computing $LenLCS(i, j)$.

Writing the recurrence equation

- If X_i and Y_j end with the same character $x_i=y_j$, the LCS must include the character. If it did not we could get a longer LCS by adding the common character.
- If X_i and Y_j do not end with the same character there are two possibilities:
 - either the LCS does not end with x_i ,
 - or it does not end with y_j
- Let Z_k denote an LCS of X_i and Y_j

X_i and Y_j end with $x_i = y_j$

X_i

x_1	x_2	$\dots$	x_{i-1}	x_i
-------	-------	---------	-----------	-------

Y_j

y_1	y_2	$\dots$	y_{j-1}	$y_j = x_i$
-------	-------	---------	-----------	-------------

Z_k

z_1	z_2	$\dots$	z_{k-1}	$z_k = y_j = x_i$
-------	-------	---------	-----------	-------------------

Z_k is Z_{k-1} followed by $z_k = y_j = x_i$ where
 Z_{k-1} is an LCS of X_{i-1} and Y_{j-1} and
 $LenLCS(i, j) = LenLCS(i-1, j-1) + 1$

X_i and Y_j end with $x_i \neq y_j$

X_i

x_1	x_2	$\dots$	x_{i-1}	x_i
-------	-------	---------	-----------	-------

X_i

x_1	x_2	$\dots$	x_{i-1}	x_i
-------	-------	---------	-----------	-------

Y_j

y_1	y_2	$\dots$	y_{j-1}	y_j
-------	-------	---------	-----------	-------

Y_j

y_j	y_1	y_2	$\dots$	y_{j-1}	y_j
-------	-------	-------	---------	-----------	-------

Z_k

z_1	z_2	$\dots$	z_{k-1}	$z_k \neq y_j$
-------	-------	---------	-----------	----------------

Z_k

z_1	z_2	$\dots$	z_{k-1}	$z_k \neq x_i$
-------	-------	---------	-----------	----------------

Z_k is an LCS of X_i and Y_{j-1} Z_k is an LCS of X_{i-1} and Y_j

$$\text{LenLCS}(i, j) = \max\{\text{LenLCS}(i, j-1), \text{LenLCS}(i-1, j)\}$$

The recurrence equation

$$\begin{aligned} lenL(i,j) &= 0 && \text{if } i=0, \text{ or } j=0 \\ lenL(i,j) &= lenL(i-1,j-1) + 1 && \text{if } i,j > 0 \text{ and } x_i = y_j \\ lenL(i,j) &= \max\{lenL(i-1,j), lenL(i,j-1)\} && \text{otherwise} \end{aligned}$$

The dynamic programming solution

- Initialize the first row and the first column of the matrix *LenLCS* to 0
- Calculate *LenLCS* (1, *j*) for *j* = 1,..., *n*
- Then the *LenLCS* (2, *j*) for *j* = 1,..., *n*, etc.
- Store also in a table an *arrow* pointing to the array element that was used in the computation.
- It is easy to see that the computation is $O(mn)$

Example

	y_j	B	D	C	A
x_i	0	0	0	0	0
A	0	↑ 0	↑ 0	↑ 0	↖ 1
B	0	↖ 1	← 1	← 1	↑ 1
C	0	↑ 1	↑ 1	↖ 2	← 2
B	0	↖ 1	↑ 1	↑ 2	↑ 2

จุดเริ่มต้น

To find an LCS follow the arrows, for each diagonal arrow there is a member of the LCS

LCS-Length (X, Y)

```
m ← length[X]
n ← length[Y]
for i ← 1 to m do
    c[i, 0] ← 0
for j ← 1 to n do
    c[0, j] ← 0
```

LCS-Length (X, Y) cont.

```
for i ← 1 to m do
  for j ← 1 to n do
    if  $x_i = y_j$ 
       $c[i, j] \leftarrow c[i-1, j-1] + 1$ 
       $b[i, j] \leftarrow \text{"D"}$ 
    else
      if  $c[i-1, j] \geq c[i, j-1]$ 
         $c[i, j] \leftarrow c[i-1, j]$ 
         $b[i, j] \leftarrow \text{"U"}$ 
      else
         $c[i, j] \leftarrow c[i, j-1]$ 
         $b[i, j] \leftarrow \text{"L"}$ 
return c and b
```

LCS Theorem

- เราสามารถสร้างขั้นตอนวิธีแบบ dynamic programming สำหรับโจทย์ *LCS* ได้ดังนี้
 - **Theorem:** With a score of 1 for each match and a zero for each mismatch or space , the matched characters in an alignment of maximum value for a *LCS*.
- เนื่องจากเป็นอัลกอริทึมแบบ dynamic programming ทั่วไป ค่าความซับซ้อนจึงอยู่ที่ $O(nm)$
- หากเปรียบเทียบกับโจทย์ สายอักขระร่วมที่ยาวที่สุด- longest substring problem -ซึ่งมีค่าความซับซ้อนอยู่ที่ $O(n+m)$ จะเห็นได้ว่าโจทย์การหา ลำดับย่อย มีความซับซ้อนกว่า สายอักขระ
- คำถาม- เราสามารถหาวิธีการที่ดีกว่านี้ได้หรือไม่ ?

S1 : **A** -- **A** **T** -- **G** **G** **C** **C** -- **A** **T** **A** **n=10**
 S2: **A** **T** **A** **T** **A** **A** **T** **T** **C** **T** **A** **T** -- **m=12**

- The optimal alignment is shown above. Note the alignment shows three insert (dark), one delete (green) and three substitution or replacement operations (blue), which gives an edit distance of 7.
- But, the 3 replacement operations can be realized by 3 insert and 3 delete operations because a replacement is equivalent to first delete the character and then insert a character in its place like:

G -- **G** -- **C** --
 -- **A** -- **T** -- **T**

- if we give a cost of 2 for **replace** operation and cost of 1 for both **insert** and **delete** operations, the minimum edit distance D can be computed in terms of the length L of LCS as:
- For the above example, $n=10$, $m=12$, $L=6$. So, $D=10$ (6 insert and 4 delete).

$$D = m - n + 2L$$

Direct Computation of *LCS* by Dynamic Programming

- More efficient although the asymptotic complexity remains the same, $O(nm)$.
- Let L denote The equations are given below without proof (which is simple).

$$L(0,0) = 0$$

$$L(i,0) = 0$$

$$L(0,j) = 0$$

$$L(i,j) = 1 + L(i-1,j-1) \text{.....} S_1(i) = S_2(j)$$

$$L(i,j) = \max[L(i,j-1), L(i-1,j)] \text{.....} S_1(i) \neq S_2(j)$$

- Again, if we leave suitable back pointers in the matrix, trace(s) can be derived for the *LCS*.

Example 3: Edit Distance

= 6 + 8 - 2*5 = 4

S ₁	=	<u>A</u>		<u>T</u>		<u>C</u>		<u>A</u>		<u>T</u>							
S ₂	=	T		A		<u>A</u>		<u>T</u>		<u>C</u>		<u>A</u>		<u>T</u>		A	
		↓		↓									↑	<u>A</u>		↓	

<i>j</i>		0	<i>T</i>	<i>A</i>	<i>A</i>	<i>T</i>	<i>C</i>	<i>A</i>	<i>T</i>	<i>A</i>
<i>i</i>			1	2	3	4	5	6	7	8
0		0	0	0	0	0	0	0	0	0
A 1		0	0	1	1	1	1	1	1	1
T 2		0	1	1	1	2	2	2	2	2
C 3		0	1	1	1	2	3	3	3	3
A 4		0	1	2	2	2	3	4	4	4
A 5		0	1	2	3	3	3	4	4	5
T 6		0	1	2	3	4	4	4	5	5

A Faster Algorithm for *LCS*

- An algorithm that is asymptotically better than $O(nm)$ for determining *LCS*.
- Implies that for special cases of edit distance, there exist more efficient algorithm.
- Definition:
 - Let Π be a set of n integers, not necessarily distinct.
- Definition:
 - An *increasing subsequence(IS)* of Π is a subsequence of Π whose values are *strictly* increasing from left to right.
- Example: $\Pi=(5,3,4,4, 9,6,2,1,8,7,10)$. $IS=(3,4,6,8,10), (5,9,10)$

- Definition:

- A *longest increasing subsequence (LIS)* of π is an *IS* π of maximum length.

- Definition:

- A *decreasing subsequence (DS)* of π is a non-increasing subsequence of π .

- Example: $DS=(5,4,4,2,1)$.

- Definition:

- A **cover** is a set of disjoint DS of Π that covers or contains all elements of Π . The size of the cover c equals the number of DS in the cover.

- Example: $\Pi=(5,3,4,9,6,2,1,8,7)$ Cover: $\{ (5,3,2,1),(4),(9,6),(8,7) \}$. $C=\# \text{ of } DS=4$.

- Definition:

- A **smallest cover (SC)** is a cover with a minimum value of c .

Determine *LIS* and *SC* simultaneously in $O(n \log n)$

- **Lemma:**

- If I is an *IS* of Π with length equal to the size of a cover C of Π , then I is a *LIS* of Π and C is the smallest cover of size c .

Proof

- If I is an increasing sequence, it cannot contain more than one element from a decreasing sequence.
- This means that no increasing subsequence can have size more than the size of any cover C , that is, if

$$C = C_1 \cup C_2 \cup \dots \cup C_k$$

a maximum of one element from each can participate in any increasing sequence.

- Thus, an IS derived from this decomposition can have a maximum length of $|C| = c$. Conversely, C must be the smallest. If not, let c' be the length of a cover C' such that $|C'| < c$ i.e., if we derive IS from C , it must contain more than one element from one of the decreasing sequence of C' , which is not possible. Hence C has to be of smallest size.

Construction of a cover

- **Greedy algorithm** to derive a cover:
 - Starting from the left of π , examine each successive number in π .
 - Append the current number at the left-most subsequence derived so far if it is possible do that maintaining the decreasing sequence property.
 - If not start a new decreasing subsequence beginning with the current element.
 - Proceed until π is exhausted.

Example

- $\pi=(5,3,4,9,6,2,1,8,7,10)$
- $D1=(5,3,2,1)$, $D2=(4)$, $D3=(9,6)$, $D4=(8,7)$, $D4=(10)$
- The algorithm has $O(n^2)$ complexity. We will present an $O(n \log n)$ algorithm.

An Efficient Algorithm for Constructing the Cover

- We use a data structure which is a list containing the last number of each of the decreasing sequence that is being constructed.
- The list is always sorted in increasing order. An identifier indicating which list the number belongs to also included.
- Procedure Decreasing Sequence Cover... x_n)
- Input: $\Pi = \langle x_1, x_2, \dots, x_n \rangle$, the list of input numbers.
- Output: the set of decreasing sequences D_i constituting the cover.

$O(n \log n)$ Algorithm

- Initialize: $i \leftarrow 1$; $D_i = (x_1)$; $L = (x_1, i)$; $j \leftarrow 1$;
- For $i=2$ to n do
 - Search the x -fields of L to find the *first* x -value such that $x_i < x$takes $O(\log n)$ time.
 - If such a value exists, then insert x at the end in the list D_i and set $x_i \leftarrow x$ in L ... This step takes *constant* time.
 - If such a value does not exist in L , then set $j \leftarrow j+1$. insert in L a new element (x, j) and start a new decreasing sequence $D_j = (x)$

End

- Lemma:

- At any point in the execution of the algorithm the list L is sorted in increasing order with respect to x -values as well as with respect to identifier value.

- In fact two separate lists will be better from practical implementation point of view.

- Theorem:

- The greedy cover can be constructed taking $O(n \log n)$ time. A longest increasing sequence and a smallest cover thus can be constructed using $O(n \log n)$ time.

Example: $\Pi=(5,3,4,9,6,2,1,8,7,10)$

i=1	x1=5	L={(5,1)}	D1=(5)	
2	3	{{(3,1)}}	(5,3)	
3	4	{{(3,1),(4,2)}}	(5,3)	D2=(4)
4	9	{{(3,1),(4,2),(9,3)}}	(5,3)	(4) D3=(9)
5	6	{{(3,1),(4,2),(6,3)}}	(5,3)	(4) (9,6)
6	2	{{(2,1),(4,2),(6,3)}}	(5,3,2)	(4) (9,6)
7	1	{{(1,1),(4,2),(6,3)}}	(5,3,2,1)	(4) (9,6)
8	8	{{(1,1),(4,2),(6,3),(8,4)}}	(5,3,2,1)	(4) (9,6) D4=(8)
9	7	{{(1,1),(4,2),(6,3),(7,4)}}	(5,3,2,1)	(4) (9,6) D4=(8,7)
10	10	{{(1,1),(4,2),(6,3),(7,4),(10,5)}}	(5,3,2,1)	(4) (9,6) D4=(8,7) D5=(10)

The x-component of the list, if separated, will look like the following during execution:

(5),(3),(3,4), (3,4,9), (3,4,6), (2,4,6),(1,4,6),
 (1,4,6,7),(1,4,6,7,10)

Reduction of *LIS* problem to *LCS* problem

- Definition:
 - Given sequences $S1$ and $S2$, let r_i be the number of occurrence of the i th character of $S1$ in $S2$.
 - Example:

(position index in sequence $S2$:) 1 2 3 4 5 6

$S1 = a \ b \ a \ c \ x$ and $S2 = b \ a \ a \ b \ c \ a$

Then, $r(1) = 3, \ r(2) = 2, \ r(3) = 3, \ r(4) = 1, \ r(5) = 0.$

- Definition:
 - for each distinct character x in $S1$, define $list(x)$ to be the positions of x in $S2$ in decreasing order.
- Example: $list(a) = (6, 3, 2)$; $list(b) = (4, 1)$, $list(c) = (5)$, $list(x) = \varnothing$ (empty sequence).

- Definition: Let $\sqcup (S1, S2)$ be a sequence obtained by concatenating *list*(*si*) for $i=1, 2, \dots, n$ where n is the length of $S1$ and si is the i th symbol of $S1$.
- Example: $\sqcup (S1, S2) = (6, 3, 2, 4, 1, 6, 3, 2, 5)$.

- Theorem:

- Every increasing sequence I of $\Pi(S1, S2)$ specifies an equal length common subsequence of $S1$ and $S2$ and vice versa. Thus a longest common subsequence LCS of $S1$ and $S2$ corresponds to a longest increasing sequence of $\Pi(S1, S2)$.

- Example: $\Pi(S1, S2) = (\underline{6}, \underline{3}, \underline{2}, \underline{4}, \underline{1}, \underline{6}, \underline{3}, \underline{2}, \underline{5})$. The possible longest increasing sequences used as indices to access the characters in $S2$ yield the LCS as: $(1, 2, 5) = b a c$, $(2, 3, 5) = a a c$, $(3, 4, 6) = a b a$ for $S1 = a b a c x$ and $S2 = b a a b c a$.